Namo

Named dimensional data for Ruby.

Namo is a Ruby library for working with multi-dimensional data using named dimensions. It infers dimensions and coordinates from plain arrays of hashes — the same shape you get from databases, CSV files, JSON, and YAML — so there's no reshaping step.

The design rests on a few stances: every hash key is a dimension and none is privileged as a coordinate or value; formulae attach to a Namo alongside data and re-evaluate on each access, appearing as derived dimensions alongside the data dimensions; operators that combine Namos all take Namos and return Namos — as do the subset-returning Enumerable methods (`select`, `reject`, `sort_by`, `uniq`, and the rest) — so analytical pipelines close; and the formula mechanism is type-agnostic — strings, dates, booleans, and arbitrary Ruby objects work as readily as numbers.

Installation

```
gem install namo
```

Or in your Gemfile:

```
gem 'namo'
```

Usage

Create a Namo instance from an array of hashes:

```
require 'namo'

sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])
```

Data may be passed positionally, as above, or by the `data:` keyword where that reads more explicitly:

```
sales = Namo.new(data: [
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}
])
```

When both are given, the positional argument wins and the keyword `data:` is ignored.

Dimensions and coordinates are inferred:

```
sales.dimensions
# => [:product, :quarter, :price, :quantity]

sales.coordinates[:product]
# => ['Widget', 'Gadget']

sales.coordinates[:quarter]
# => ['Q1', 'Q2']
```

Every key is a dimension; every value is a coordinate. There's no schema declaration and no choosing which column is "the index" — `price` and `quantity` are no less first-class than `product` and `quarter`.

Selection

Select by named dimension using keyword arguments:

```
# Single value
sales[product: 'Widget']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Multiple dimensions
sales[product: 'Widget', quarter: 'Q1']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}
# ]>

# Range
sales[price: 10.0..20.0]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Array of values
sales[quarter: ['Q1']]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
# ]>

# Proc predicate
sales[price: ->(v){v < 20.0}]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Regex predicate
sales[product: /^W/]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

Procs receive the dimension value and select the row when they return truthy. They handle arbitrary predicates — multi-condition tests, nil-aware checks, anything Ruby can express — and compose with everything else:

```
sales[price: ->(v){v < 20.0}, quantity: ->(v){v > 100}]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

Regexes match against the dimension value coerced with `to_s`, so they work against strings, symbols, numbers, dates, or anything else with a sensible string form. `nil` becomes `""` — `//` matches it, `/./` doesn't.

```
sales[product: /widget/i]                          # case-insensitive
sales[product: /Widget|Gadget/]                    # alternation
sales[product: /^W/, quarter: 'Q1']                # mixed with exact
```

Procs and regexes mix freely with exact values, arrays, ranges, projection, and contraction in the same `[]` call.

Projection

Project to specific dimensions:

```
sales[:product, :price]
# => #<Namo [
#   {product: 'Widget', price: 10.0},
#   {product: 'Widget', price: 10.0},
#   {product: 'Gadget', price: 25.0},
#   {product: 'Gadget', price: 25.0}
# ]>
```

Selection and projection can be chained:

```
sales[product: 'Widget'][:quarter, :price]
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[:quarter, :price, product: 'Widget']
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Contraction

Contraction is the complement of projection. Projection says "keep these dimensions"; contraction says "remove these dimensions, keep everything else":

```
sales[-:price, -:quantity]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'},
#   {product: 'Gadget', quarter: 'Q1'},
#   {product: 'Gadget', quarter: 'Q2'}
# ]>
```

The `-:price` syntax uses unary minus on Symbol to produce a negated dimension. Mixing projection and contraction in the same call is an error — the two modes are mutually exclusive:

```
sales[:product, -:price]  # => ArgumentError
```

Selection and contraction can be chained:

```
sales[product: 'Widget'][-:price, -:quantity]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[-:price, -:quantity, product: 'Widget']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'}
# ]>
```

Selection, projection, and contraction always return a new Namo instance, so everything chains.

Concatenation

+ is the first of Namo's binary operators: it takes a Namo on each side and returns a Namo. The same shape holds for −, &, |, ^, ==, ===, <, <=, >, >= and (later) the composition operators — Namo in, Namo (or boolean) out — so analytical pipelines stay queryable end-to-end.

+ combines two Namo objects that share the same dimensions by appending the rows of the second to the first:

```
q1_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

q2_sales = Namo.new([
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

all_sales = q1_sales + q2_sales
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
#   {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match — the same dimension names in the same order — or the operator raises an `ArgumentError`. Two Namos holding the same columns in a different order must be normalised to a common column order before they can be combined. Formulae carry through from the left-hand side.

Row Removal

− removes from the first Namo any row that appears exactly in the second:

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

discontinued = Namo.new([
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])
```

```
sales - discontinued
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

Removal is exact — every dimension, every value must match. The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Intersection

`&` returns the rows present in both Namo objects, like `Array#&`:

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

confirmed = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

sales & confirmed
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Union

`|` returns all rows from both sides, deduplicated, like `Array#|`:

```
q1_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

all_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
])

q1_sales | all_sales
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Symmetric Difference

^ returns rows that appear in one side but not both:

```
set_a = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

set_b = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
])

set_a ^ set_b
# => #<Namo [
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Composition

∗ is the equi-join operator. It pairs rows from two Namos where coordinates match on every shared dimension, like an inner join on the shared dimension names:

```
ohlcv = Namo.new([
  {symbol: 'BHP', date: '2025-01-01', close: 42.5},
  {symbol: 'RIO', date: '2025-01-01', close: 118.3}
])

fundamentals = Namo.new([
  {symbol: 'BHP', pe: 14.5},
  {symbol: 'RIO', pe: 9.2}
])

ohlcv ∗ fundamentals
# => #<Namo [
#   {symbol: 'BHP', date: '2025-01-01', close: 42.5, pe: 14.5},
#   {symbol: 'RIO', date: '2025-01-01', close: 118.3, pe: 9.2}
# ]>
```

Inner-join semantics: unmatched rows from either side are dropped. Output dimensions are `self.data_dimensions` followed by `other.data_dimensions` exclusive to other. Duplicates on shared coordinates are preserved multiplicatively — output multiplicity is the product of input multiplicities on each matching key.

The two Namos must have at least one shared data dimension. No overlap raises an `ArgumentError` — the asymmetry with ∗∗ is deliberate, and falling through to a Cartesian product would silently turn a logic error into a large pile of nonsense rows. Formulae merge from both sides; the left-hand side wins on conflict. A name that is data on one side and a formula on the other also raises an `ArgumentError` — the operands disagree about what the name means, with no last-write order to appeal to — so resolve before composing: `audited[-:margin] ∗ modelled`.

Conditional join ∗ takes an optional block that decides which matched rows to pair with each left row. Without a block, every shared-dimension match pairs, as above. With one, the block is handed the current

left row and the right rows already matched on the shared dimensions, and returns the subset to pair — the refinement plain `*` can't express, because it pairs every match.

The canonical case is matching each daily price to a single quarterly report — the most recent one dated on or before it. Plain `*` pairs *every* matching quarter; the block narrows that to the one the matching rule picks.

```
prices = Namo.new([
  {symbol: 'BHP', date: '2025-02-15', close: 42.5},
  {symbol: 'BHP', date: '2025-05-20', close: 44.0}
])

quarterly = Namo.new([
  {symbol: 'BHP', quarter_end: '2024-12-31', eps: 1.0},
  {symbol: 'BHP', quarter_end: '2025-03-31', eps: 1.2}
])

prices.*(quarterly) do |row, candidates|
  candidates[quarter_end: ->(qe){qe <= row[:date]}].sort_by{|f| f[:quarter_end]}.last(1)
end
# => #<Namo [
#   {symbol: 'BHP', date: '2025-02-15', close: 42.5, quarter_end: '2024-12-31', eps: 1.0},
#   {symbol: 'BHP', date: '2025-05-20', close: 44.0, quarter_end: '2025-03-31', eps: 1.2}
# ]>
```

`row` is the `Row` for the current left row, carrying self's formulae, so `row[:date]` and any self formula resolve inside the block. `candidates` is a Namo of the shared-dimension matches, carrying other's formulae, so the block can select on other's derived dimensions too. The block returns a Namo of the rows to pair: one row for the single-match rule above, though it may return zero, one, or many — it's a selector, not a reducer. An empty returned Namo pairs nothing, so that left row is dropped, preserving inner-join semantics. The block can also be passed as a named proc, `prices.*(quarterly, &most_recent_quarter)`.

The block changes only which rows pair. Formulae carry through exactly as in the no-block form — other's merged under self's, self winning on conflict — and the rows the block returns contribute data only.

Cartesian product

`**` is the Cartesian product. Every row from the left paired with every row from the right:

```
products = Namo.new([{product: 'Widget'}, {product: 'Gadget'}])
quarters = Namo.new([{quarter: 'Q1'}, {quarter: 'Q2'}])

products ** quarters
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'},
#   {product: 'Gadget', quarter: 'Q1'},
#   {product: 'Gadget', quarter: 'Q2'}
# ]>
```

Output has `self.data.length * other.data.length` rows. Output dimensions are `self.data_dimensions + other.data_dimensions`, in operand order. Duplicates are preserved multiplicatively.

The two Namos must have **no** shared data dimensions — the precondition is the mirror image of `*`. Any overlap raises an `ArgumentError`; allowing it would produce rows with the same dimension named twice. Formulae merge from both sides; the left-hand side wins on conflict, and a data/formula name collision between the operands raises, as for `*`.

The visual relationship is intentional: ∗ is the filtered version, ∗∗ is the explosive version — more sigil, more output.

Conditional product ∗∗ takes an optional block on the same contract. Where ∗'s block receives the rows pre-matched on the shared dimensions, ∗∗'s receives *all* of other's rows — there are no shared dimensions to match on — and returns the subset to pair with each left row.

This expresses a conditional product: pair each order with only the shipping tiers that can carry it.

```
orders = Namo.new([
  {order: 'A', weight: 5},
  {order: 'B', weight: 15}
])

tiers = Namo.new([
  {tier: 'light', max_weight: 10},
  {tier: 'heavy', max_weight: 20}
])

orders.**(tiers) do |row, candidates|
  candidates[max_weight: ->(w){w >= row[:weight]}]
end
# => #<Namo [
#   {order: 'A', weight: 5, tier: 'light', max_weight: 10},
#   {order: 'A', weight: 5, tier: 'heavy', max_weight: 20},
#   {order: 'B', weight: 15, tier: 'heavy', max_weight: 20}
# ]>
```

The contract matches ∗'s: `row` carries self's formulae, `candidates` carries other's, the block returns a Namo of rows to pair, and an empty return drops the left row. A block that returns its `candidates` unchanged reproduces the no-block product exactly — ∗∗ is its own block form with the identity selector, just as ∗ is ∗∗ with the shared-dimension match applied first.

Decomposition

`/` removes from the left Namo the dimensions that are also in the right, then dedupes the projected rows. It's the inverse of ∗ and ∗∗:

```
combined = Namo.new([
  {symbol: 'BHP', date: '2025-01-01', close: 42.5, pe: 14.5},
  {symbol: 'RIO', date: '2025-01-01', close: 118.3, pe: 9.2}
])

fundamentals = Namo.new([
  {symbol: 'BHP', pe: 14.5},
  {symbol: 'RIO', pe: 9.2}
])

combined / fundamentals
# => #<Namo [
#   {date: '2025-01-01', close: 42.5},
#   {date: '2025-01-01', close: 118.3}
# ]>
```

The intersection of dimensions — here `:symbol` and `:pe` — is removed. Everything else stays. The projected rows are deduplicated, so `/` answers "what's left when these dimensions are factored out?" rather than "what

rows survive a column drop?". Formulae carry through from the left-hand side.

`/` has no precondition. When the two Namos share no dimensions, the intersection is empty, nothing is removed, and `self / other` returns a Namo equal to self:

```
shipments = Namo.new([{order_id: 1, weight: 10}])
weather = Namo.new([{date: '2025-01-01', temperature: 22}])

shipments / weather
# => #<Namo [{order_id: 1, weight: 10}]> — equal to shipments
```

The round-trip identity holds for the ∗∗ case exactly:

```
a = Namo.new([{symbol: 'BHP'}, {symbol: 'RIO'}])
b = Namo.new([{quarter: 'Q1'}, {quarter: 'Q2'}])

(a ** b) / b == a
# => true
```

For ∗, the round-trip is lossy on the dimensions that were shared between the operands:

```
a = Namo.new([{symbol: 'BHP', close: 42.5}, {symbol: 'RIO', close: 118.3}])
b = Namo.new([{symbol: 'BHP', pe: 14.5}, {symbol: 'RIO', pe: 9.2}])

(a * b) / b
# => #<Namo [{close: 42.5}, {close: 118.3}]>
# Equal to a[-:symbol]. :symbol was shared and is lost.
```

The asymmetry is inherent: `/` operates only on the two values it receives and can't distinguish "shared dimension that belonged to both" from "exclusive dimension that belonged only to the right". Removing the intersection is the only rule expressible from the operands alone, and it gives clean recovery from ∗∗ and well-defined (if lossy) recovery from ∗.

Why `/` is loose ∗ and ∗∗ raise when their preconditions are violated — combining unrelated Namos has no natural answer, and silently producing arbitrary output would turn a logic error into a large pile of nonsense rows. `/` is different: it's a projecting operator, not a combining one, and projecting away nothing returns the original. The no-precondition rule isn't a fallback; it's the structurally correct result.

This earns `/` three properties a strict version would lose:

- **Identity test.** `combined / other == combined` exactly when the two have no shared dimensions — answers "are these Namos dimensionally independent?" without explicit introspection. Same shape as `a & b == a` answering subset from 0.6.0.
- **Idempotence.** `(c / b) / b == c / b`. Once `b`'s dimensions are removed, removing them again does nothing.
- **Pipeline composition.** A processing step that applies `/ separator` can run over any Namo regardless of whether the separator's dimensions apply. Uninvolved Namos pass through unchanged; involved Namos get stripped. The pipeline doesn't need to special-case applicability.

This is the same pattern that makes `Array#-` useful with arrays that aren't subsets: `[1, 2, 3] - [9] == [1, 2, 3]`, not an error. The no-op-on-non-applicable behaviour lets the operator compose into pipelines that don't know in advance whether the operation applies.

Equality

Comparison on Namos is **multiset-theoretic on rows**: the comparison operators — `==`, `eql?`, `===`, `<`, `<=`, `>`, `>=` — ignore row order, so two Namos holding the same rows in a different order compare equal, while row multiplicities count (they *are* data). That stance is shared across the equality, pattern-match, and subset/superset operators documented below. Row order is otherwise preserved: `to_h` and `values` depend on

it for columnar alignment, and `each`, `first`, `last`, `take`, `drop`, and `+` all observe it. It is the comparison operators alone that treat the sequence of rows as a multiset.

`==` is multiset equality on rows. Class and formulae are ignored; row order is ignored; row multiplicities are not.

```
a = Namo.new([{x: 1}, {x: 2}])
b = Namo.new([{x: 2}, {x: 1}])

a == b
# => true

a == Namo.new([{x: 1}, {x: 1}, {x: 2}])
# => false
```

`eql?` is stricter: it also requires the class to match and the formula names to match. Like `===`, it ignores proc bodies — proc identity isn't a meaningful equivalence in Ruby (`proc{...} == proc{...}` is false), so neither `===` nor `eql?` uses it.

`hash` is consistent with `eql?` and is content-based, so equal Namos hash equally and can be used as Hash keys:

```
h = {a => 'first'}
h[b]
# => 'first'
```

`equal?` is unchanged from Ruby's default — it tests object identity.

`===` answers a different question: does the candidate have the same dimensions and the same formula names? Row data is ignored, and so are the proc bodies themselves — only the names matter. This is the `===` semantics that case statements use, so Namos can serve as templates for analytical shape:

```
sales_shape = Namo.new([{product: 'X', quarter: 'Q1', price: 0.0, quantity: 0}])
sales_shape[:revenue] = proc{|row| row[:price] * row[:quantity]}

q1 = Namo.new([{product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}])
q1[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales_shape === q1
# => true (same dimensions, same formula name)

sales_shape == q1
# => false (different rows)
```

The two `:revenue` procs are independently-written and not the same object — `proc{...} == proc{...}` is false in Ruby. But `===` doesn't compare proc identity; it asks "do these Namos have the same analytical shape?" and the shape is the set of dimensions plus the set of formula names.

Each comparison operator answers a distinct question: `eql?` is strictest (class + data + formula names); `==` is data identity; `===` is analytical identity; the subset operators are data containment.

Rows participate in value semantics on the same data-only basis. `Row#==`, `Row#eql?`, and `Row#hash` compare the underlying row hash and ignore the surrounding Namo's formulae — two Rows with the same data are equal regardless of which Namo yielded them. This makes Rows usable as Hash keys and Set members, and underwrites whole-row deduplication on the Enumerable side:

```
a = Namo.new([{x: 1}]).first
b = Namo.new([{x: 1}]).first
a == b           # => true
a.eql?(b)        # => true
```

```
{a => :found}[b] # => :found
```

The omission of `Row#===` and Row-level `</<=/>/>=` is deliberate: a Row is a record, not a collection, so the set-theoretic operators don't translate. The value-semantics trio (`==`, `eql?`, `hash`) is what a hash-shaped value needs to behave correctly in Ruby's collection machinery; that's the whole Row-comparison story.

Subset and Superset

`<`, `<=`, `>`, `>=` are multiset subset and superset relations on rows.

```
small = Namo.new([{x: 1}, {x: 2}])
large = Namo.new([{x: 1}, {x: 2}, {x: 3}])

small <= large
# => true

small < large
# => true

large > small
# => true
```

Equal sets are `<=` and `>=` each other, but neither `<` nor `>`. Disjoint sets are none of the above — unless one side is empty, in which case it is a subset of (and disjoint with) the other.

Multiplicity matters: a single `{x: 1}` is a proper subset of two `{x: 1}`s.

```
one = Namo.new([{x: 1}])
two = Namo.new([{x: 1}, {x: 1}])

one < two
# => true
```

The dimensions must match; different dimensions raise an `ArgumentError`. Comparing against a non-Namo raises a `TypeError`.

Formulae

Define computed dimensions using `[]=`:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales[:product, :quarter, :revenue]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]>
```

Formulae aren't materialised into row data — they re-evaluate on every access. A `:revenue` value reflects the current `:price` and `:quantity` at the moment you ask for it, so derived values stay in sync with whatever the underlying data is doing.

Formulae compose:

```
sales[:cost] = proc{|row| row[:quantity] * 4.0}
sales[:profit] = proc{|row| row[:revenue] - row[:cost]}
```

```
sales[:product, :quarter, :profit]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', profit: 600.0},
#   {product: 'Widget', quarter: 'Q2', profit: 900.0},
#   {product: 'Gadget', quarter: 'Q1', profit: 840.0},
#   {product: 'Gadget', quarter: 'Q2', profit: 1260.0}
# ]>
```

Formulae work with selection and projection:

```
sales[product: 'Widget'][:revenue, :quarter]
# => #<Namo [
#   {revenue: 1000.0, quarter: 'Q1'},
#   {revenue: 1500.0, quarter: 'Q2'}
# ]>
```

Formulae carry through selection — a filtered Namo instance remembers its formulae.

Projection of derived dimensions Naming a derived dimension in a projection asks for its values: they are computed against the source and stored in the result's rows, and the formula is dropped — the name is a data dimension of the result. Omitting it carries the formula live, recomputing from the result's own rows on every access:

```
sales[:price, :quantity, :revenue].derived_dimensions
# => [] — :revenue is stored values, a snapshot taken at projection

sales[:price, :quantity].derived_dimensions
# => [:revenue] — :revenue recomputes from the projected rows on every access
```

The projection list is the selector: name a derived dimension for a snapshot, omit it to keep it as computation. A carried formula whose inputs the projection dropped breaks on access — the same caveat as contracting away a formula's inputs.

Cross-row formulae A formula's arity selects its calling convention. A proc with **one** parameter receives the row, as above. A proc with **two** parameters receives `(row, namo)`, where `namo` is the Namo the row belongs to — so the formula can reach beyond the current row to the rest of the collection. That's what cross-row computation needs: moving windows, ranks, running totals, anything whose value depends on the row's neighbours.

A simple moving average reads the surrounding rows through `namo`:

```
prices = Namo.new([
  {symbol: 'AAA', date: 1, close: 10.0},
  {symbol: 'AAA', date: 2, close: 20.0},
  {symbol: 'AAA', date: 3, close: 30.0}
])

prices[:sma] = proc{|row, namo|
  window = namo[symbol: row[:symbol], date: ->(d){d <= row[:date]}]
  window.values(:close).sum / window.count.to_f
}

prices.values(:sma)
# => [10.0, 15.0, 20.0]
```

`namo` is the Namo that yielded the row, live — so the window always reflects the current state of the object you ask through. A filtered Namo's rows window over the filtered rows; an operator result's rows window

over the result. Appending a row changes every cross-row value on the next access, with no caching.

One-arity formulae are unchanged, and the two forms mix freely — a one-arity formula can reference a two-arity one, and a two-arity formula can reference a one-arity one, by name.

Resolving a two-arity formula needs a Namo to window over. A `Row` constructed directly, without one, raises an `ArgumentError` naming the formula rather than letting the missing context surface as an unrelated error.

Parameterised formulae A formula can declare parameters beyond `(row, namo)`. The arguments arrive at access time, through `Row#[]`, so one definition serves every column and every setting:

```
prices[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: row[:symbol], date: ->(d){d <= row[:date]}].last(period)
  window.sum{|r| r[field]} / window.count.to_f
end

prices.last[:sma, :close, 20]    # 20-period moving average of close
prices.last[:sma, :volume, 50]   # 50-period moving average of volume
```

The number of *required* parameters decides a formula's calling convention. One means row-scoped, two or more means collection-scoped, and everything past the second receives the arguments given at the call site. A trailing splat or optional after `(row, namo)` makes the arguments optional — `proc{|row, namo, *fields|}` accepts any number, including none. A proc whose second parameter is optional (`->(row, namo = nil){...}`) requires only one, so it stays row-scoped.

Argument counts are enforced. Asking with the wrong number — too few for the formula's parameters, too many for a fixed-arity proc, or any at all for a data dimension or an unparameterised formula — raises an `ArgumentError` stating the counts, rather than letting `nil` flow into the formula body:

```
prices.last[:sma]        # ArgumentError: wrong number of arguments for :sma (given 0, expected 2)
prices.last[:close, 20]    # ArgumentError: wrong number of arguments for :close (given 1, expected 0)
```

A formula that requires arguments can't be materialised without them. `values(:sma)`, `coordinates(:sma)`, naming `:sma` in a projection, and selecting on it all raise the same `ArgumentError`; the no-argument `values`, `coordinates`, and `to_h` omit the dimension, returning everything that can be materialised. `dimensions` and `derived_dimensions` still list it — it is queryable, with arguments. To materialise particular values, bind the arguments in a one-arity wrapper and ask for that:

```
prices[:sma_close_20] = proc{|row| row[:sma, :close, 20]}
prices[:date, :sma_close_20]   # materialises per the usual projection rule
```

Removing a formula `detach(:name)` removes a formula, returns the Namo, and chains — `sales.detach(:revenue).deta` Detaching a name that isn't there is a quiet no-op, so a cleanup step composes over any Namo without first checking whether the formula is present. It is the sanctioned way to drop a derived dimension, sitting over the store-level primitive `formulae.delete`.

`detach` and contraction are complements across two axes. Contraction (`sales[-:price]`) is non-mutating — it returns a *new* Namo with a *data* dimension removed and the formulae carried. `detach` mutates the receiver and removes a *derived* dimension. So the pair spans both the data/derived split and the algebra/mutation split: contraction is algebra over data, `detach` is mutation of the derived side. `detach` never touches `@data` — naming a data dimension is a no-op and the column survives; removing a data dimension is contraction's job.

Formularies A *formulary* is a reusable body of derived dimensions — a module whose public instance methods are formulae. A Namo attaches a formulary, and from then on those methods resolve as derived dimensions through the same interface as `[]=` formulae. The methods take `row` (and `namo`, and any further parameters) exactly as a `[]=` formula does:

```
module OrderFlow
  include Namo::Formulary

  def signed_volume(row)
    row[:buys] - row[:sells]
  end

  def cum_delta(row, namo)
    namo[date: ..row[:date]].values(:signed_volume).sum
  end

  private

  def net(row)
    row[:buys] + row[:sells]
  end
end

flows = Namo.new([
  {date: 1, buys: 60, sells: 40},
  {date: 2, buys: 80, sells: 120},
  {date: 3, buys: 75, sells: 75}
])

flows.attach(OrderFlow)
flows.values(:signed_volume)   # => [20, -40, 0]
flows.values(:cum_delta)       # => [20, -20, -20]
flows.derived_dimensions       # => [:signed_volume, :cum_delta]
```

`<<` appends a *constituent*. For a base Namo the constituents are formularies and rows: a **module** attaches (the operator form of `attach`, so `flows << OrderFlow` reads the same as `flows.attach(OrderFlow)`); a **Hash** appends as a data row; a **Row** (one drawn from another Namo) appends its underlying hash as a row. It returns the Namo, so the arms chain freely: `flows << OrderFlow << {date: 4, buys: 10, sells: 5} << Scoring`. Two things `<<` deliberately does not take: a bare callable (formula *registration* stays with `[]=`, which dispatches callable-vs-scalar and enforces data/formula exclusivity) and a whole Namo (combining collections is `+`'s job) — both raise. A bare Hash is therefore always a row, never a body of formulae. Appending a row whose keys collide with an existing formula name **raises** an `ArgumentError` naming the collision — a name is data or derived, never both. The guard is symmetric with `attach`'s: `[]=` names one column at the call site, so its scalar form evicts a same-named formula as your explicit intent, but a row is a bulk append you didn't name column-by-column, so the safe response is to refuse rather than let data and a formula of the same name disagree on access. Contract or rename first, then append.

A module brands itself a formulary by including `Namo::Formulary`. The marker is mandatory — `attach` raises `ArgumentError` for an untagged module, so an ordinary module of helpers can't be mistaken for a body of derivations. Within a formulary, the public methods are the derivations and `private` methods are helpers: `net` above never appears in `derived_dimensions` and never resolves as a dimension. The separation needs no per-method declaration — public or private says it.

Attaching **copies** the formulary's public methods into the Namo's formula collection — each becomes a stored formula, indistinguishable from one assigned through `[]=`. From there they are `[]=` formulae in everything but how they were defined: they resolve through the same path, carry through the same operators, and obey the same exclusivity rule.

The copy is a *snapshot* taken at the moment of attachment, and this is where a formulary departs from ordinary Ruby. A real `include` (or `extend`) is a live link: its methods resolve through the ancestor chain, so a method added to the module later, a redefinition, or a further module mixed in afterwards are all seen by

objects that already include it. `attach` forgoes that — it copies the method set once, so later changes to the module are not reflected on an already-attached Namo. Re-attach to pick them up. (A live-link mechanism may come in a later release; it is not in 0.23.0.)

Two ways to give a Namo a formulary. `attach` (and `<<`) adds one to an individual Namo at runtime, as above. Alternatively, `include` the formulary into a `Namo` subclass — `class TradingAnalysis < Namo; include OrderFlow; end` — and every instance picks it up when it is built. The rule that separates them is simple: *the class `include` is honoured once, when each instance is constructed; everything after that is `attach`.* Both copy (snapshot), so a formulary `included` into the class **after** an instance exists won't reach that instance — `attach` to it, or build a fresh one. A one-off plain `Namo` has no class to `include` into per-instance, so it always takes `attach`. Where both apply, ordinary last-write-wins holds: a runtime `attach` (or `[]=`) overrides a class-included formula of the same name on that instance, and the most-recently `included` formulary wins among the class's own.

Because each method is copied into the store, the rules already governing `[]=` apply:

- **Most-recent wins.** Attaching a second formulary that defines a colliding name overwrites the first, exactly as a second `[]=` would. Equally, `[]=` and `attach` interleave as plain last-write-wins on a shared name — whichever was applied last governs.
- **Data collisions raise.** A name is data or derived, never both. If a formulary method's name collides with an existing data column, attaching raises an `ArgumentError` naming the collision rather than silently destroying the data. The reason it raises where `[]=` silently clears: `[]=` names one column at the call site, so replacing it is your explicit intent; a formulary names a whole *module*, so a data collision is a name you never typed — and possibly several columns at once — that you most likely didn't mean to destroy. The same raise guards the class-`include` channel, where it fires at construction. Resolve it explicitly first — contract the column away (`namo[-:signed_volume]`) or rename — then attach.
- **Materialise-and-drop on projection.** Naming a formulary method in a projection snapshots its values into a data column *and drops the formula*, exactly as for a `[]=` formula: `flows[:date, :signed_volume]` returns a Namo reporting `:signed_volume` as data, not derived, with all access paths agreeing on the stored snapshot.
- **Carry-through.** Selection, the set and composition operators, and a projection that *omits* the name all keep the formula live, computing against the result's own rows — the carry-through is free, because the formulae are ordinary store entries.

`attach!` is the forceful sibling of `attach`. Where `attach` refuses a data collision, `attach!` **evicts** the colliding data columns first — deleting each one exactly as assigning a proc through `[]=` clears a same-named data column — then attaches. The bang is consent: `attach` won't destroy a column you never named, but `attach!` is you saying you meant to. The motivating case is refreshing a formulary over a *materialised snapshot* — once a derived dimension has been projected into a stored column (materialise-and-drop, above), re-attaching the formulary that defined it collides with the now-data column, and `attach!` re-attaches over it in one step where `attach` would require you to contract the column away first. `<<` stays the guarded form: a module through `<<` routes to `attach`, so the operator never evicts — only an explicit `attach!` call does.

Note that there is no counterpart in `[]=`'s scalar-over-formula eviction and is worth naming: a formulary method that reads a column its own name evicts will **recurse** on access. The eviction removes the datum, and exclusive storage leaves no shadowed value behind the formula, so `Row#[]` re-enters the formula rather than falling back to data — a `def close(row); row[:close] * ...; end` attached with `attach!` over a `:close` data column has no `:close` data left to read. `[]=` rarely bites this way because you write the proc at the call site, beside the name it replaces; a formulary is authored elsewhere, so a method reading its own evicted column is easy to attach without noticing. Self-reference is unguarded here as everywhere in the formula mechanism — keep a formulary's inputs and outputs under distinct names, or `attach` (not `attach!`) and contract deliberately.

`detach(OrderFlow)` is the reverse of `attach`: it removes the formulary's public method names from the store, so `flows.attach(OrderFlow)` then `flows.detach(OrderFlow)` leaves the formulae as they started. It works by name and keeps no provenance — the store is a snapshot, so `detach` removes *whatever currently holds each name*, including a `[]=` formula that later overwrote a formulary method of the same name. Like

the `attach` side, the marker is required: `detach` raises `ArgumentError` for an untagged module, so you can only detach what could have been attached. The mutating family is `attach` / `attach!` / `detach`; `<<` has no removal counterpart (it *appends* constituents — a removal operator would be cuteness over clarity), and there is no `detach!` because removing a formula can't collide with data, so there is no guard for a bang to force through.

Two Namos that expose the same names — one via `[]=`, one via a formulary — are `===`; a Namo with a formulary attached is not `===` to a vanilla Namo of the same data dimensions.

Formulary methods take `row` explicitly and index it (`row[:buys]`), the same as `[]=` formulae. Resolving bare names inside a method body (`buys` for `row[:buys]`) and memoising derived values on a frozen Namo arrive in a later release; until then a formulary method is a plain function of its `row` (and `namo`).

Polymorphic `[]=`

`[]=` dispatches on the value assigned. A callable — anything that responds to `call`, so a proc, a lambda, or a `Method` — registers a formula, as above. Anything else broadcasts the value to every row:

```
sales[:status] = 'active'
sales.values(:status)
# => ['active', 'active', 'active', 'active']

sales[:revenue] = proc{|row| row[:price] * row[:quantity]}
sales.values(:revenue)
# => [1000.0, 1500.0, 1000.0, 1500.0]
```

The two branches mirror the polymorphism `[]` already has on the selection side, where a single bracket call dispatches over exact values, arrays, ranges, procs, and regexes. Rather than introduce a separate `broadcast` or `set_all` method for the scalar case, `[]=` reads the same way for both: `sales[:status] = 'active'` says "set status to active across this Namo," and `sales[:revenue] = proc{…}` says "derive revenue from each row."

The two branches enforce **exclusive storage**: a name is either a data dimension or a derived dimension, never both. Assigning a callable clears any data column of that name; assigning anything else clears any formula of that name. The last write wins, and there is no shadowing:

```
sales[:x] = 5                       # :x is a broadcast data value
sales[:x] = proc{|row| row[:price]} # :x is now a formula — the broadcast value is gone

sales[:x] = proc{|row| row[:price]} # :x is a formula
sales[:x] = 5                     # :x is now a broadcast data value — the formula is gone
```

Exclusivity ties directly to the inspection vocabulary: a name assigned a scalar shows up in `data_dimensions`; a name assigned a proc shows up in `derived_dimensions`; never in both, so it appears in `dimensions` exactly once.

Only a callable takes the formula branch. An array doesn't respond to `call`, so it broadcasts as the per-row value rather than registering as a formula:

```
sales[:weights] = [1, 2, 3]
sales.values(:weights)
# => [[1, 2, 3], [1, 2, 3], [1, 2, 3], [1, 2, 3]]
```

Coordinates and values

`dimensions` covers the *queryable namespace* — every name you can ask for, whether it lives in the row data or is computed by a formula. Once formulae are defined, they appear alongside data dimensions:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
sales.dimensions
# => [:product, :quarter, :price, :quantity, :revenue]

sales.data_dimensions
# => [:product, :quarter, :price, :quantity]

sales.derived_dimensions
# => [:revenue]
```

`coordinates` gives the unique values per dimension, including derived ones:

```
sales.coordinates[:product]
# => ['Widget', 'Gadget']

sales.coordinates[:revenue]
# => [1000.0, 1500.0]
```

`values` gives the full per-row sequence — duplicates preserved, row order preserved:

```
sales.values[:product]
# => ['Widget', 'Widget', 'Gadget', 'Gadget']

sales.values[:revenue]
# => [1000.0, 1500.0, 1000.0, 1500.0]
```

Both `coordinates` and `values` accept positional arguments. With no args they return a Hash across the queryable namespace; with one arg they lazily compute and return just that column as an Array; with multiple args they return a subset Hash containing just the requested columns:

```
sales.values(:product)
# => ['Widget', 'Widget', 'Gadget', 'Gadget']

sales.values(:product, :quarter)
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2']
# }

sales.coordinates(:revenue)
# => [1000.0, 1500.0]
```

Single-arg access is lazy: `sales.values(:revenue)` evaluates the formula only across the rows of `:revenue`, without materialising the other columns. The bracket form (`sales.values[:revenue]`) still works through ordinary Hash lookup but pays for the full materialisation up front.

`coordinates` is `values` with `.uniq` applied per column — `coordinates(dim) == values(dim).uniq` holds for every dimension.

`to_h` is the Ruby-conventional alias for the full `values` Hash:

```
sales.to_h
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
#   price: [10.0, 10.0, 25.0, 25.0],
#   quantity: [100, 150, 40, 60],
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
# }
```

Unknown dimensions propagate `nil` per row — `values(:missing)` returns `[nil, nil, ...]` rather than raising or returning a sentinel, matching the convention used by `Row#[]` and `[]` selection. Use `dimensions.include?(:dim)` if you need to check membership directly.

Enumerable

Namo includes `Enumerable`, so `each`, `reduce`, `map`, `select`, `min_by`, and all the rest work out of the box. Rows are yielded as `Row` objects, so formulae are accessible during enumeration:

```
sales.reduce(0){|sum, row| sum + row[:quantity]}
# => 350

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:quantity]}
# => 250

sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales.reduce(0){|sum, row| sum + row[:revenue]}
# => 5000.0

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:revenue]}
# => 2500.0

sales.map{|row| row[:product]}
# => ['Widget', 'Widget', 'Gadget', 'Gadget']

sales.min_by{|row| row[:price]}[:product]
# => 'Widget'

sales.flat_map{|row| [row[:price]]}
# => [10.0, 10.0, 25.0, 25.0]
```

The subset-returning Enumerable methods — `select`, `reject`, `sort_by`, `first(n)`, `last(n)`, `take`, `drop`, `take_while`, `drop_while`, `uniq`, and `partition` — return Namos rather than Arrays (`partition` returns `[Namo, Namo]`), carrying formulae through. This keeps the analytical chain closed: the result of a filter is still selectable, projectable, and operable, exactly like the operators that combine Namos:

```
sales.select{|row| row[:price] < 20.0}.values(:price).sum
# => 20.0

sales.select{|row| row[:price] < 20.0}[product: 'Widget'][:quarter, :revenue].to_a
# => [{quarter: 'Q1', revenue: 1000.0}, {quarter: 'Q2', revenue: 1500.0}]
```

Without an argument, `first` and `last` return a single `Row` (or `nil` on an empty Namo), following Ruby's convention; with an argument they return a Namo of that many rows. `uniq` dedupes on full-row equality (`Row#==`), or on a block's return value when given one. `select`'s aliases `filter` and `find_all` follow the override and return Namos too.

The transforming and reducing methods are deliberately left as Enumerable's defaults, because their results aren't row-shaped and so can't be a Namo: `map`/`collect` and `flat_map` return Arrays of whatever the block produces; `reduce`/`inject`, `sum`, `count`, `min_by`, and `max_by` return scalars. `each` is unchanged — it yields Rows, or returns an Enumerator with no block.

Extracting data

`to_a` returns an array of hashes — the row-oriented form:

```
sales[:product, :quarter, :revenue].to_a
# => [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]
```

`to_h` returns a hash of arrays — the columnar form (see Coordinates and values above):

```
sales[:product, :quarter, :revenue].to_h
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
# }
```

Named Namos

A Namo can carry a name, passed by the `name:` keyword and read or set through the `name` accessor:

```
sales = Namo.new(data: rows, name: :sales)
sales.name
# => :sales

sales.name = :renamed
```

A name defaults to `nil`, and operator results are name-less by design — the result of `+`, `*`, `select`, and the rest is a derived object, not the original, so giving it the parent's name would mislead:

```
(sales + more).name
# => nil
```

This `nil`-on-derivation behaviour is what lets subclasses with side effects in `initialize` guard those effects on the name. Operator-derived instances are name-less and skip the side effects; explicitly constructed instances pass `name:` and the side effects fire:

```
class TradingAnalysis < Namo
  def initialize(positional_data = nil, data: [], formulae: {}, name: nil)
    super
    return unless name
    register_indicators
  end
end
```

`super` with no parentheses forwards every argument — positional and keyword — to `Namo#initialize` unchanged. The `return unless name` guard means a subclass need not override every operator to stop the result of `*` or `select` from re-running its construction side effects: it guards on `name` instead.

Collections

`Namo::Collection` is a hierarchical aggregate — a Namo that holds an Array of named Namos (its `members`) and exposes summary and detail views across them. It is the first member of the Namo family beyond `Namo` itself.

The motivating case is a hierarchical budget. Each sub-assembly of a car (`powertrain`, `chassis`, `body`, …) is a Namo with shared columns; the whole car is a `Collection` of those sub-assemblies, queryable both at summary level ("weight by assembly") and detail level ("every line item across all assemblies"):

```
class Car < Namo::Collection
  def summary(dimension, by: :assembly, reducer: :sum)
    super
  end

  def detail(by: :assembly)
    super
  end
end

class SubAssembly < Namo; end

powertrain = SubAssembly.new(name: :powertrain, data: [
  {component: 'engine', weight: 200, cost: 50000},
  {component: 'gearbox', weight: 80, cost: 20000}
])
chassis = SubAssembly.new(name: :chassis, data: [{component: 'frame', weight: 150, cost: 30000}])
body   = SubAssembly.new(name: :body, data: [{component: 'panels', weight: 60, cost: 15000}])

gt = Car.new
gt << [powertrain, chassis, body]

gt.summary(:weight).to_a
# => [
#   {assembly: :powertrain, weight: 280},
#   {assembly: :chassis, weight: 150},
#   {assembly: :body, weight: 60}
# ]

gt.summary(:weight).values(:weight).sum   # total weight by summing the assembly summaries
# => 490

gt.detail.values(:weight).sum              # total weight by summing every line item
# => 490
```

`Car` overrides `summary`/`detail` only to set `by: :assembly` as the per-class default and then calls `super`. A bare `Namo::Collection.new` works equally well, defaulting `by:` to `:member` and taking it at the call site.

Lazy detail, behaving as its line items A Collection's substance is its `members`; the inherited `@data` is a *derived view* of them. Any inherited row-operation — selection, projection, `each`, `values`, the set and composition operators — reads that view, so a Collection transparently behaves as its **detail** (the lossless union of its members' rows). Nothing has to be called first:

```
gt.values(:weight)
# => [200, 80, 150, 60]

gt[component: 'engine'].values(:cost)
# => [50000]
```

Detail is the lazy view because a Collection's rows simply *are* its members' rows; a summary is a reduction you pose against them, so it is never reached by accident — only through `summary` or `as_summary`.

Four view methods The views come in a non-mutating pair and a mutating pair:

- `summary(dimension, by:, reducer:)` and `detail(by:)` are **non-mutating** — each returns a fresh

Namo derived from the members, leaving the Collection untouched. Use these when you want a view to keep: assign the result to a variable and operate on it independently.
- `as_summary(dimension, by:, reducer:)` and `as_detail(by)` are **mutating** — each sets the Collection's data to the chosen view and returns `self`, for a fluent step. (`as_detail` carries no `dimension`, so its label argument is positional: `as_detail(:assembly)`.)

```
gt.summary(:cost, reducer: :mean)          # a fresh Namo; gt is unchanged
gt.as_summary(:weight)                     # gt's data becomes the summary; returns gt
gt.as_detail(:assembly)                    # gt's data becomes the detail; returns gt
```

`reducer:` is any method the member's column responds to — `:sum` (the default) and `:mean` are typical (`:mean` via a statistics gem that adds `Array#mean`).

Inject-iff-absent `detail(by:)` unions the members' rows and labels each with its origin, but only when that label isn't already present:

- If `by` is **already a dimension** in a member's rows, the row passes through untouched — the dimension is intrinsic.
- If `by` is **not** present, `detail` injects it (`row.merge(by => member.name)`), promoting the member's name into a dimension.

This single conditional is where assembly (`<<`, members named extrinsically) and partition (`group_by`, members named by an intrinsic value — 0.20.0) meet. For an assembled Collection, `as_detail(:assembly)` is the dimension-creating step: it promotes the member name into real data and **retains** it. From then on the structure is intrinsic and round-trips are exact; the promoted dimension is removed only by explicit contraction (`gt[-:assembly]`), never automatically.

`<<` and unnamed members `<<` accepts a single member or an array of them. A member whose `name` collides with an existing member's **replaces** it (last-write-wins), making the name → member mapping a dictionary rather than a multimap:

```
gt << SubAssembly.new(name: :powertrain, data: [...])  # replaces the existing :powertrain
gt << [front_suspension, rear_suspension]                # adds each
```

On a Collection `<<` dispatches to the *constituent* appropriate to a collection: a **member** (a Namo, or an array of them) is added as above; a **module** attaches a formulary (a Collection is a Namo, so it carries formulae over its detail view — see below); a loose **Hash or Row raises** an `ArgumentError` that redirects to member-add. This is the one place `Namo#<<` and `Collection#<<` deliberately diverge: a base Namo appends a loose row, a Collection refuses it — a Collection's rows are *derived* from its members, so a loose row has no durable home (the next member-add rebuilds the data view and would erase it). The honest response is to refuse and point at member-add. A whole Namo arriving at a Collection is unambiguously a member, never a row-merge, because `<<` has no whole-Namo arm (that is `+`).

There is no insertion-time guard against unnamed members. An unnamed member is simply appended (no name to collide on) and is unfindable by `find` — the honest consequence of having no name, not an error. `find(name)` returns the member with that name, or `nil`:

```
gt.find(:chassis)    # => the chassis SubAssembly
gt.find(:missing)    # => nil
```

(`find(name)` is member lookup; it shadows `Enumerable#find` on Collections. Predicate search over rows remains available as `detect`.)

Collection formulae A formulary attached to a Collection resolves over its detail rows like any other — a Collection is a Namo, and its data view is the materialised detail. A *collection formula* needs no new mechanism: it is an ordinary two-arity formulary method whose `namo` argument is the Collection, reaching for a collection view (`summary`, `detail`, `members`) in its body. It resolves through the same `derive` path as every other formula; member-awareness lives in the method body, not in any marker or second store. By

convention, name that argument `namo_collection` to signal it expects a Collection — a plain Namo lacking `members/summary` will `NoMethodError`, which is honest self-enforcement, no guard needed.

```
module FleetMetrics
  include Namo::Formulary

  def member_count(row, namo_collection)
    namo_collection.members.size
  end
end

gt << FleetMetrics
gt.values(:member_count)   # => one entry per detail row, each the member count
```

View lifetime and liveness Materialisation is pure-live: the Collection rebuilds its data view from the current members on every `<<`, with no memoisation. So a mutation is reflected immediately — add a member, then summarise or detail, and the new member is included.

A mutating `as_summary/as_detail` view **persists until the next `<<`**, which re-materialises detail. So `as_summary` is for "be the summary for this immediate chain":

```
gt.as_summary(:weight).values(:weight)    # => [280, 150, 60]   (the summary)
gt << front_suspension                     # re-materialises detail
gt.values(:weight)                       # => [200, 80, 150, 60, ...]   (line items again)
```

Freeze-gated memoisation is a 2.x optimisation — opt-in via `freeze`, transparent, and never changing this observable behaviour. `group_by` (0.20.0) is the partition-side constructor for the same type: it splits a Namo into a `Collection`, the mirror of assembling one with `<<`.

Partitioning with `group_by` `group_by(dimension)` is the partition-side constructor for a `Collection` — the mirror of assembling one with `<<`. It splits a Namo into one member per distinct value of the dimension, each a Namo holding that group's rows, named by its group value:

```
prices.group_by(:symbol)
# => #<Namo::Collection members: [:BHP, :RIO, :CBA]>

prices.group_by(:symbol).summary(:close, reducer: :mean)
# => Namo with {member:, close:} rows — mean close per symbol
```

The grouping dimension is retained in every member — the split runs *along* the axis, it doesn't consume it — so the partition inverts exactly through `as_detail` on the same dimension:

```
prices.group_by(:symbol).as_detail(:symbol) == prices
# => true
```

Because data and derived dimensions are treated alike, you can group by a formula as readily as by a stored column. Grouping by a derived dimension materialises it first — the grouped-by formula becomes a stored value in each member and is dropped, while every other formula carries through live:

```
prices[:value_score] = proc{|r| r[:pe] < 10 ? 2 : r[:pe] < 15 ? 1 : 0}
prices.group_by(:value_score)
# => one member per score; :value_score is now a data column in each
```

This gives a single inversion law over the whole namespace — `namo.group_by(d).as_detail(d) == namo[*namo.data_dimensions, d]` for any `d`, with the exact-original round-trip being the data-dimension instance of it. A nil-valued group produces a nil-named member, holding its rows and round-tripping like any other.

Name

Namo: nam(ed) (dimensi)o(ns). A companion to Numo (numeric arrays for Ruby). And in Aussie culture ‘o’ gets added to the end of names.

Contributing

1. Fork it (https://github.com/thoran/namo/fork)
2. Create your feature branch (git checkout -b my-new-feature)
3. Commit your changes (git commit -am ‘Add some feature’)
4. Push to the branch (git push origin my-new-feature)
5. Create a new pull request

License

MIT